Documentação Técnica Integral: COMINDIR05

Implementação do Algoritmo RAFT

Projeto de Sistemas Distribuídos

12 de novembro de 2026

Conteúdo

1 Introdução e Visão Geral **2**

2 Resumo Integral da Implementação RAFT **2**

3 Estrutura dos Projetos e Ficheiros **2**
- 3.1 Projeto `raft04-no` . . . 2
- 3.2 Projeto `raft04-produtor` . . . 2
- 3.3 Projeto `raft04-processador` . . . 2
- 3.4 Ficheiros criados por cada classe e o seu propósito . . . 2

4 Compilação e Execução dos Projetos **3**
- 4.1 Compilação . . . 3
- 4.2 Ordem de Execução . . . 3

5 RaftCheck e CausalCheck **3**
- 5.1 RaftCheck . . . 3
- 5.2 CausalCheck . . . 4

6 Cenários de Execução e Resultados Esperados **4**

7 Análise Individual de Todas as Classes **4**
- 7.1 Wire . . . 4
- 7.2 VoteRequest e VoteReply . . . 4
- 7.3 RaftState . . . 5
- 7.4 HeartbeatMonitor . . . 5
- 7.5 Batch . . . 5
- 7.6 RecordFile . . . 5
- 7.7 Producer . . . 6
- 7.8 AlarmProcessor . . . 6
- 7.9 PartitionNode . . . 6

8 Análise Detalhada de `tratarEntrada` e Registo de IGNORADO **7**

9 Fluxo Completo de Execução **7**

10 Conclusão **7**

1 Introdução e Visão Geral

O sistema implementa o algoritmo de consenso RAFT para resolver o problema de um líder fixo que, ao falhar, bloqueava indefinidamente a partição de dados. Para tal, introduz grupos de três nós idênticos por partição, capazes de eleger um novo líder democraticamente e gerir termos de liderança.

O projeto divide-se em três aplicações:

- **Produtor:** Lê sensores e publica leituras no tópico IPFS.
- **Nós de Partição:** Três réplicas que elegem um líder e replicam os dados.
- **Processador:** Ouve os lotes, funde os relógios e gera alarmes.

2 Resumo Integral da Implementação RAFT

A inicialização arranca com os nós a lerem os ficheiros de estado. Cada nó inicia um `RaftState` como SEGUIDOR no termo gravado. Uma thread verifica a cada 200ms se o prazo de eleição expirou. Se expirar, o termo é incrementado, o nó vota em si próprio e passa a CANDIDATO, publicando um `PEDIDO_VOTO`.

Os restantes nós avaliam o pedido com base na restrição de voto (o candidato tem de ter o log mais atualizado). Se a maioria (2 de 3) votar a favor, o candidato torna-se LIDER e inicia a publicação de `LOTE`s a cada 2 segundos.

Se um líder falhar, os seguidores deixam de receber os LOTEs. Após o timeout, é iniciada uma nova eleição com um termo superior. O antigo líder, ao voltar, verá o termo superior e demitir-se-á.

3 Estrutura dos Projetos e Ficheiros

3.1 Projeto `raft04-no`

Contém a lógica central. Ficheiros: `PartitionNode.java`, `RaftState.java`, `Batch.java`, `RecordFile.java`, `VoteRequest.java`, `VoteReply.java`, `SensorRecord.java`, `VectorClock.java`, `Wire.java`, `IpfsPubSub.java`, `Partitioner.java`.

3.2 Projeto `raft04-produtor`

Contém `Producer.java`, `HeartbeatMonitor.java` e as classes comuns.

3.3 Projeto `raft04-processador`

Contém `AlarmProcessor.java`, `HeartbeatMonitor.java` e as classes comuns.

3.4 Ficheiros criados por cada classe e o seu propósito

Esta é uma parte crucial do sistema, pois o estado persistente e os logs são a única forma de auditar a segurança e recuperar de falhas.

- `RaftState`: Cria o ficheiro `estado-p<k>-n<i>.txt`.
  - *Formato:* `termo;votouEm`.
  - *Propósito:* Garantir a persistência do termo e do voto. Se o nó falhar e reiniciar, lê este ficheiro para não voltar a votar no mesmo termo (evita *split-brain*).

- `PartitionNode`: Cria 4 ficheiros essenciais por cada nó.
  - `no-p<k>-n<i>.txt`: Registo dos dados (`seq;termo;sensor;temp;instante;L;V`). É o ficheiro que é truncado se houver divergências.
  - `eventos-p<k>-n<i>.txt`: Log de eventos de estado (`CANDIDATO`, `VOTO`, `LIDER`, `TRUNCADO`, `IGNORADO`). Usado pelo `RaftCheck`.
  - `hb-p<k>-n<i>.txt`: Histórico de heartbeats enviados (`termo;hb;instante;...`). Prova que o líder estava vivo.
  - `truncados-p<k>-n<i>.txt`: Cópia dos dados apagados na truncagem (auxiliar ao `RaftCheck`).
- `Producer`: Cria 2 ficheiros.
  - `produtor.txt`: Registo de todos os `IN`s enviados com sucesso (`instante;p<k>;sensor;temp`). O `RaftCheck` usa isto para calcular as leituras perdidas.
  - `monitor-produtor.txt`: Log do `HeartbeatMonitor` (SUSPEITA, NOVO_LIDER, HB_PERDIDO).
- `AlarmProcessor`: Cria 1 ficheiro.
  - `monitor-processador.txt`: Log do `HeartbeatMonitor` e avisos de alarmes gerados para partições sem líder.

4 Compilação e Execução dos Projetos

4.1 Compilação

```
mkdir -p raft04-no/out raft04-produtor/out raft04-processador/out
javac -d raft04-no/out raft04-no/src/raft04/*.java
javac -d raft04-produtor/out raft04-produtor/src/raft04/*.java
javac -d raft04-processador/out raft04-processador/src/raft04/*.java

```

4.2 Ordem de Execução

1. **IPFS Daemon:** `ipfs daemon`.
2. **Nós de Partição (9 processos):**

```
java -cp raft04-no/out raft04.PartitionNode 0 1 3
java -cp raft04-no/out raft04.PartitionNode 0 2 3
java -cp raft04-no/out raft04.PartitionNode 0 3 3
# Repetir para particoes 1 e 2

```

3. **Processador:** `java -cp raft04-processador/out raft04.AlarmProcessor 3`
4. **Produtor:** `java -cp raft04-produtor/out raft04.Producer 3`

5 RaftCheck e CausalCheck

5.1 RaftCheck

Verifica a segurança da eleição lendo os logs. Conta os termos com mais de um líder (deve ser 0), verifica se os termos voltaram atrás, calcula a indisponibilidade e conta as leituras perdidas (cruzando o `produtor.txt` com os ficheiros `no-*`).

5.2 CausalCheck

Verifica a integridade dos relógios lógicos. Confirma se $V[k] = seq$ em todos os registos de todos os nós e se o relógio de Lamport respeita a causalidade entre partições.

Aspeto	**RaftCheck**	**CausalCheck**
Objetivo	Validar segurança e disponibilidade RAFT	Validar causalidade e Lamport
Observa	Logs, eleições, hb, produtor	Ficheiros de registos (`no-*`)
Deteta	Dois líderes, leituras perdidas, truncagem	Quebras de V[k]=seq ou Lamport

6 Cenários de Execução e Resultados Esperados

Cenário	**Ação**	**Comportamento Real do Sistema**	**Verificação**
Arranque	Iniciar 9 nós	Líder eleito no termo 1. Os outros 2 votam.	RaftCheck: 1 termo, 0 votos divididos.
Falha do Líder	Matar o líder (kill -9)	Indisponibilidade de 6-12s (tempo de eleição). Os seguidores elegem um novo líder no termo 2.	RaftCheck mede o tempo e leituras perdidas nesse intervalo.
Líder Isolado	Isolar o líder (ficheiro isolar)	O líder isolado **continua a receber e a gravar INs no disco**, mas não os publica. A maioria elege novo líder (termo 2). Ao voltar, o antigo líder trunca os dados que gravou no isolamento.	RaftCheck: Evento TRUNCADO no antigo líder. Dois líderes em termos diferentes.
Falha de Maioria	Matar 2 nós	O nó sobrevivente (se seguidor) incrementa o termo infinitamente mas nunca é eleito (falta quórum). O sistema fica 100% indisponível para escritas.	RaftCheck: Múltiplos termos sem líder.

7 Análise Individual de Todas as Classes

7.1 Wire

Identificação: Projeto No, Produtor e Processador.
Responsabilidade: Guardar constantes globais e gerar nomes de tópicos/portos.
Métodos: `PERIODO_HB_MS` (2000ms), `ELEICAO_MIN/MAX_MS`, `isolated(k,i)`.

7.2 VoteRequest e VoteReply

Identificação: Projeto No.
Responsabilidade: DTOs para as mensagens de eleição.
Métodos: `encode()` e `decode(String)`.

7.3 RaftState

Identificação: Projeto No.
Responsabilidade: O cérebro lógico da eleição.
Atributos: `term`, `votedFor` (persistentes); `role`, `leader`, `votes` (voláteis).

Método `persist()`:

```
    private void persist() {
        try {
            Path tmp = stateFile.resolveSibling(stateFile.getFileName() + ".
tmp");
            Files.writeString(tmp, term + ";" + votedFor + "\n",
StandardCharsets.UTF_8);
            Files.move(tmp, stateFile, StandardCopyOption.REPLACE_EXISTING,
StandardCopyOption.ATOMIC_MOVE);
        } catch (IOException e) {
            throw new UncheckedIOException(e);
        }
    }

```

Método `onVoteRequest()`:

```
    public synchronized Vote onVoteRequest(long cTerm, int candidate, long
cLastSeq, long cLastTerm,
    long myLastSeq, long myLastTerm) {
        if (cTerm > term) adoptTerm(cTerm);
        boolean upToDate = cLastTerm > myLastTerm
        || (cLastTerm == myLastTerm && cLastSeq >= myLastSeq);
        boolean granted = cTerm == term
        && (votedFor == 0 || votedFor == candidate)
        && upToDate;
        if (granted) {
            votedFor = candidate;
            persist();
            resetTimer();
        }
        return new Vote(term, granted);
    }

```

7.4 HeartbeatMonitor

Identificação: Produtor e Processador.
Responsabilidade: Detetar falhas do líder (timeout 6s).
Método `onHeartbeat()`: Ignora termos obsoletos, regista `NOVO_LIDER` se o termo subir.

7.5 Batch

Identificação: Projeto No, Produtor e Processador.
Responsabilidade: Transportar o LOTE (`term`, `leader`, `lastSeq`, `lastTerm`).

7.6 RecordFile

Identificação: Projeto No.
Responsabilidade: Escrever no disco.
Método `rewrite()`:

```
    public synchronized void rewrite(List<SensorRecord> registos) throws
IOException {
```

```
            StringBuilder sb = new StringBuilder();
            for (SensorRecord r : registos) {
                sb.append(r.toLine()).append(System.lineSeparator());
            }
            Path tmp = path.resolveSibling(path.getFileName() + ".tmp");
            Files.writeString(tmp, sb.toString(), StandardCharsets.UTF_8);
            Files.move(tmp, path, StandardCopyOption.REPLACE_EXISTING,
    StandardCopyOption.ATOMIC_MOVE);
        }

```

7.7 Producer

Identificação: Produtor.
Responsabilidade: Publicar `IN`.
Método `publicar()`: Regista no `produtor.txt` apenas se o publish for bem-sucedido.

7.8 AlarmProcessor

Identificação: Processador.
Responsabilidade: Consumir LOTEs e gerar alarmes.
Método `receber()`: Filtra termos antigos e deduplica com `seq:termo`.

7.9 PartitionNode

Identificação: Projeto No.
Responsabilidade: Orquestrar rede, ficheiros e delegar ao `RaftState`.

Método `tornarLider()`:

```
        private void tornarLider() {
            if (raft.termIfLeader() == -1) return;
            seq = ultimoSeq();
            if (registos.isEmpty()) {
                lamport = 0;
                vector = new VectorClock(n);
            } else {
                SensorRecord ultimo = registos.get(registos.size() - 1);
                lamport = ultimo.lamport;
                vector = ultimo.vector.copy();
            }
            hb = 0;
            pendentes = new ArrayList<>();
        }

```

Método `executarRecuperacao()`:

```
        if (d <= ultimoSeq()) {
            List<SensorRecord> cauda = new ArrayList<>(registos.subList((int) d
    - 1, registos.size()));
            registos.subList((int) d - 1, registos.size()).clear();
            ficheiro.rewrite(registos);
            evento("TRUNCADO;p" + k + ";de=" + d + ";descartados=" + cauda.size
    () + ...);
        }

```

8 Análise Detalhada de `tratarEntrada` e Registo de IGNORADO

O método `tratarEntrada` é invocado quando chega um `IN`.

O que acontece se o nó não for líder:

```
        synchronized (this) {
            long t = raft.termIfLeader();
            if (t == -1) {
                evento("IGNORADO;p" + k + ";IN " + sensor + " " + temp + " (sem
    lideranca)");
                return;
            }
        }

```

Impacto no SSD:

- **Frequência:** Numa rajada de 1000 leituras, os 2 nós seguidores escrevem 1000 linhas de `IGNORADO` em disco cada um (2000 escritas).
- **Desgaste:** Cada escrita abre o ficheiro, aloca espaço e fecha, degradando a performance da JVM e submetendo o SSD a um ciclo de escrita massivo.
- **Mitigação:** Deveria ser usado um `BufferedWriter` com flush periódico, ou *rate-limiting* (escrever 1 log por segundo), para não matar o SSD.

9 Fluxo Completo de Execução

10 Conclusão

A implementação do RAFT no COMINDIR05 introduz uma solução robusta para falhas do líder. O sistema garante a segurança (nunca há dois líderes num termo) mas prioriza a consistência (CP) em caso de partições de rede. A principal limitação identificada é a degradação de performance/SSD causada pelo logging síncrono excessivo de eventos `IGNORADO`.

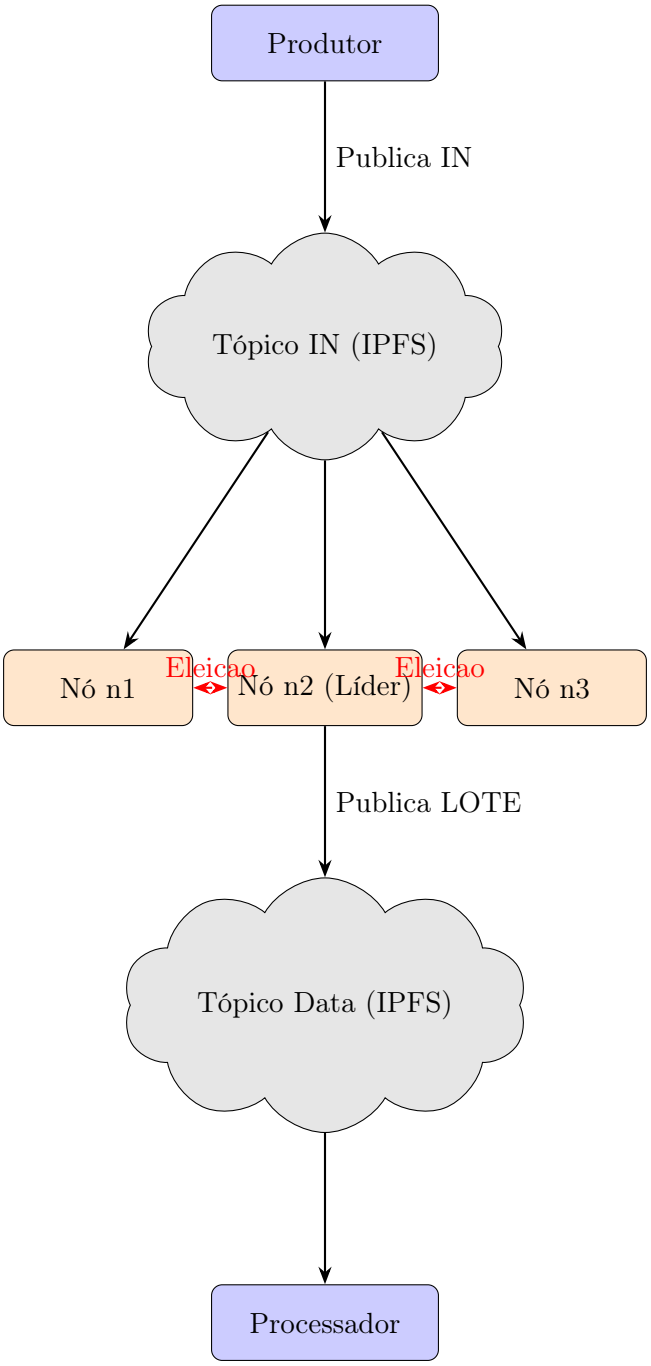


Figura 1: Arquitetura de Comunicação do Sistema

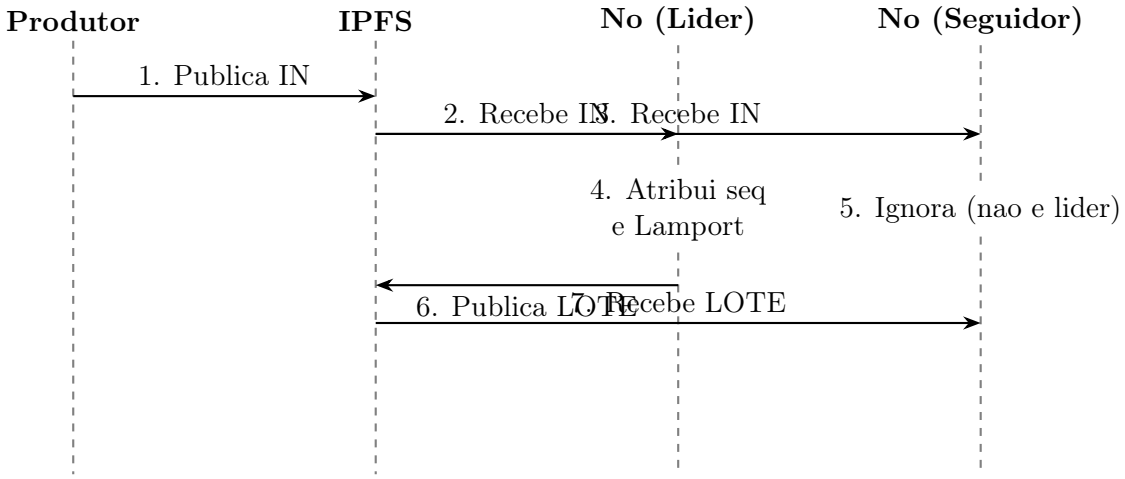


Figura 2: Fluxo de Execução de uma Leitura